

Model-Based and Data-Driven Robust Control of a Self-Balancing Robot using Passivity

Shrishgovind Umesh Revankar and Shirantha Welikala

Department of Electrical and Computer Engineering

Stevens Institute of Technology

Hoboken, NJ, USA.

{srevanka1, swelikal}@stevens.edu

Abstract—This project proposes a hybrid control workflow for a two-wheeled self-balancing robot. First, we use an existing robust model-based control system, which is simulated in MATLAB. Here we can test the model under various dynamic conditions, including collisions and weight additions. Once the model is tested under various input conditions, it will yield a large output sample. These data can be processed to create a data matrix for developing the data-driven control system. With known input-output data, the robot is able to better react to any input in real life. There are 2 types of data-driven control systems: Model-Based and Model-Free. We will conduct experiments on both systems to empirically evaluate which system demonstrates superior performance in power efficiency, computational speed, and space optimization. Passivity-based control employs convex energy shaping to achieve stability by enforcing passivity constraints and ensuring closed-loop energy dissipation during trajectory tracking. To enhance robustness, we will experiment with passivity-based control to achieve stability with minimal energy consumption.

Index Terms—Self-Balancing Robot, Model-Learning, Data Driven Control, Passivity, Dynamic Conditions.

I. INTRODUCTION

Self-balancing two-wheeled robots offer superior maneuverability with zero turning radius and compact design, outperforming multi-wheeled platforms in confined spaces like warehouses, factories, and service environments. However, their nonlinear, underactuated, and intrinsically unstable dynamics challenge control under disturbances such as terrain variations or payloads, where traditional linearized or PID methods fail due to model inaccuracies and simulation-to-real-world gaps.

Robust ADRC baseline model, which is simulated in MATLAB, is where we can tune the stability. Active Disturbance Rejection Control (ADRC) utilizes an Extended State Observer (ESO) to estimate and compensate for total disturbances, including model uncertainties, friction, and external loads in real-time, without requiring precise parameterization, providing our initial stabilizing model under nominal conditions [1].

Gaussian Process learning for data-efficient dynamics, for self-balancing robots, GPs iteratively refine simulation-to-real gaps by learning difference models between simulator and hardware trajectories. PILCO optimizes policies via long-term cost gradients, accounting for uncertainty propagation, achieving balance with minimal real trials. This yields accurate, low-bias models for data generation [3].

Model-free data-driven control, the data matrix generated is used for Model-Free Control (MFC), which uses pointwise input-output data to drive error to zero via ϕ , a smooth approximation of sign for chattering reduction. Data matrices from GP rollouts enable direct tuning via Lyapunov-stable gains [2].

Passivity enforcement via frequency-domain convex optimization for positive-real stability. It ensures dissipative, safe operation without precise models. Data-driven design uses frequency-response data from experiments, parameterizing a fixed-structure controller. Ensures stability/robustness without models; iterative refinement converges locally [5].

II. PROBLEM DESCRIPTION

Two-wheeled self-balancing robots are nonlinear, underactuated, and inherently unstable systems, modeled as wheeled inverted pendulums as stated in [1]. The dynamics are given by

$$(M + m)\ddot{x} - m\ddot{\theta} \cos \theta + m\dot{\theta}^2 \sin \theta = F, \quad (1)$$

$$l\ddot{\theta} - \ddot{x} \cos \theta - g \sin \theta = 0, \quad (2)$$

where M , m , and l denote cart mass, body mass, and pendulum length, while x , θ , and F denote position, tilt, and wheel force, respectively. Linearization around $\theta = 0$ yields unstable poles, which are sensitive to parameters [1].

The nonlinear dynamics of the two-wheeled self-balancing robot Eq.(1) and Eq.(2) are integrated into a structured control architecture. Here, the plant—with wheel force as input and states $(x, \dot{x}, \theta, \dot{\theta})$ serves as the physical subsystem, while IMU-measured θ and $\dot{\theta}$ feed into a nonlinear controller. This controller generates stabilization effort, distributed via allocation and motor stages to close the loop, abstracting the coupled cart-pendulum dynamics.

A. Model-Based Limitations

Model-based control assumes an accurate mathematical model of the plant, but for highly nonlinear, unstable systems like inverted pendulums and self-balancing robots this is hard to obtain because of unmodeled dynamics, parameter uncertainty, and time-varying conditions, which can degrade performance or even destabilize the closed loop [2] [4].

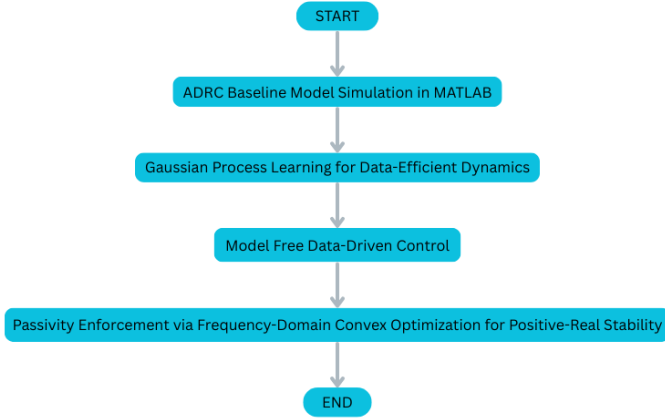


Fig. 1. Project Flowchart

B. Data-Driven Control Limitations

Data-driven control provides an appealing alternative to model-based design, but it also faces several structural limitations. First, most data-driven frameworks require persistently exciting input–output or state trajectories; if the data do not excite all relevant modes, the underlying identification or synthesis LMIs can become infeasible and stability guarantees may not hold. Second, because controller design depends directly on measured signals, data-driven schemes are typically sensitive to measurement noise and disturbances; noisy data can shrink the certified dissipativity or stability margins or even invalidate the feasibility of the corresponding matrix inequalities [4].

III. EXPECTED CONTRIBUTIONS

With each step of the project explained with the help of the flowchart in Fig. 1, we would like to discuss the goals of each step and how they contribute.

1) *Robust ADRC Baseline Model – Simulation in MATLAB*: We first build and test a classical, model-based controller (ADRC) for the self-balancing robot in MATLAB. The goal is to have a stabilizing controller and a reasonably accurate simulation model that can keep the robot upright under nominal conditions [1] [6].

2) *Gaussian Process Learning for Data-Efficient Dynamics*: Next, you use Gaussian Processes to learn a better dynamics model from a combination of simulation data and a small amount of real robot data. The goal is to capture the true (possibly nonlinear, uncertain) dynamics more accurately than the hand-derived model, while using as little real data as possible [3].

3) *Input-Output Data Matrix*: Using the GP-based model and/or experiments, you generate and store input–output trajectories in a data matrix. The goal is to create a clean dataset that describes how the robot responds to commands, without requiring an explicit parametric model [3].

4) *Model-Free Data-Driven Control*: With that data matrix, you design a controller directly from data, not from equations. The goal is to obtain a model-free controller that can swing up and stabilize/balance the robot, robust to nonlinearities and disturbances [2].

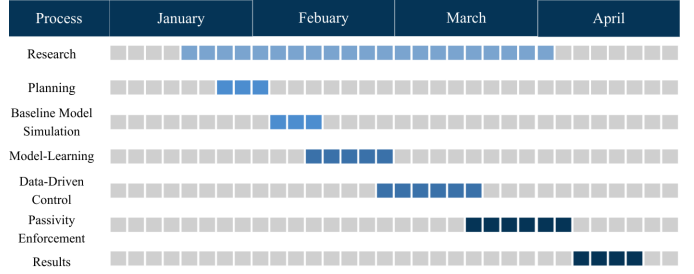


Fig. 2. Project Timeline

5) *Passivity Enforcement via Frequency-Domain Convex Optimization for Positive-Real Stability*: Finally, you refine the data-driven controller by imposing passivity constraints using convex optimization in the frequency domain. The goal is to guarantee that the closed-loop robot behaves in an energy-dissipative, stable, and safe way, even under dynamic conditions and interaction [5].

6) *Integral Benefits*: At the end of the pipeline, you have a hybrid control solution: it starts from a model-based ADRC design, learns/improves dynamics from data, builds a model-free controller, and then enforces passivity to certify robustness and safety.

IV. PROJECT TIMELINE

The Gantt chart Fig. 2, displays our plan spanning from January to April. Baseline Model Simulation will be taking a model that was previously proven. Gaussian Process learning is then used for data-efficient dynamics, in which almost all the scenarios will be simulated to get the input–output data matrix sample as large as possible. The input–output data matrix is then used to develop the data-driven control for the real robot. We will conduct experiments on different systems to empirically evaluate and determine which demonstrates superior performance in power efficiency, computational speed, and space optimization. Then we will be adding passivity in order add robustness and efficiency to the robot. The flow of the entire project will be as shown in Fig. 1.

REFERENCES

- [1] A. Lima-Perez, J. Díaz-Tellez, V. Gutierrez-Vicente, J. Estevez-Carreón, J. Perez-Perez, R. S. García-Ramírez, and J. Chavez-Galan, “Robust control of a two-wheeled self-balancing mobile robot,” in *2021 International Conference on Mechatronics, Electronics and Automotive Engineering (ICMEAE)*, 2021, pp. 196–201, doi: 10.1109/ICMEAE55138.2021.00038.
- [2] A. Baciú and C. Lazar, “Data-driven control for swing-up and stabilization of an inverted pendulum system,” in *29th Mediterranean Conference on Control and Automation (MED)*, Bari, Italy, 2021, pp. 1155–1160, doi: 10.1109/MED51440.2021.9480190.
- [3] E. Li, H. Feng, H. Zhou, X. Li, Y. Zhai, S. Zhang, and Y. Fu, “Model learning for two-wheeled robot self-balance control,” in *IEEE Intl. Conf. on Robotics and Biomimetics*, Dali, China, 2019, pp. 1582–1587.
- [4] S. Welikala, Z. Song, H. Lin, and P. J. Antsaklis, “Model-based and data-driven hierarchical control and topology co-design for robust networked systems,” in *Proc. of the American Control Conference*, 2022.
- [5] S. S. Madani and A. Karimi, “Data-driven passivity-based current controller design for power-electronic converters of traction systems,” in *59th IEEE Conf. on Decision and Control*, Jeju Island, Republic of Korea, 2020, pp. 842–847, doi: 10.1109/CDC42340.2020.9303797.
- [6] V. M. Senthilkumar, M. Siva Ramkumar, P. Velraj Kumar, T. Kamalavalli, V. G. Pratheep, K. S., and M. Sivaramkrishnan, “Model-based controller design for automatic two wheeled self balancing robot,” in *Proceedings of the Intl. Conference on Inventive Computation Technologies*, 2023, pp. 1418–1425, doi: 10.1109/ICICT57646.2023.10134094.